TTT-TSF: Test-Time Training Layers for Time Series Forecasting

Simple Auto Research

Abstract

Transformer-based models have achieved remarkable success in long-term time series forecasting (LTSF), yet their quadratic-complexity self-attention mechanism remains a bottleneck. Meanwhile, Test-Time Training (TTT) layers—a recently proposed class of sequence modeling layers whose hidden state is a machine learning model updated via self-supervised learning—have shown promise as efficient alternatives to attention in language modeling. In this work, we propose **TTT-TSF**, the first application of TTT layers to time series forecasting. Building on the inverted Transformer architecture of iTransformer, TTT-TSF replaces self-attention with TTT-Linear layers that adaptively learn cross-variate dependencies through a self-supervised reconstruction objective. Extensive experiments on 7 benchmark datasets demonstrate that TTT-TSF achieves superior performance compared to iTransformer and other state-of-the-art methods, winning 26 out of 28 settings with an overall 3.1% MSE reduction. Notable improvements are observed on Traffic (-4.6%), ECL (-4.4%), and Weather (-3.7%), with larger gains on higher-dimensional datasets, while maintaining linear complexity in sequence length.

1 Introduction

Long-term time series forecasting (LTSF) is a fundamental problem in data mining with broad applications ranging from energy management and weather prediction to traffic optimization and financial planning. The task requires models to capture complex temporal patterns and cross-variate dependencies from historical observations to predict future values.

Recent years have witnessed a paradigm shift from classical statistical methods to deep learning approaches for LTSF. Transformer-based models [Vaswani et al., 2017] have become the dominant architecture, with numerous variants proposed to improve forecasting performance [Wu et al., 2021, Zhou et al., 2022, Wu et al., 2023, Nie et al., 2023]. Notably, iTransformer [Liu et al., 2024] introduced an inverted design that treats each variate as a token and applies attention across variates rather than time steps, achieving state-of-the-art results on many benchmarks. However, a fundamental question remains: *is self-attention the best mechanism for modeling cross-variate dependencies in time series?*

Concurrently, in the sequence modeling community, new architectures have emerged as alternatives to Transformers. State space models (SSMs) like Mamba [Gu and Dao, 2024] offer linear complexity but have limited expressiveness in their hidden states. More recently, Sun et al. [2024] proposed **Test-Time Training (TTT) layers**—a novel class of sequence modeling layers where the hidden state is itself a machine learning model (e.g., a linear model), and the update rule is a step of self-supervised learning. This design allows TTT layers to maintain linear complexity while achieving expressive hidden states that can adapt to the data at hand.

We hypothesize that TTT layers are particularly well-suited for time series forecasting because: (1) cross-variate correlations in real-world time series are complex and can shift over time due to non-stationarity; (2) TTT's self-supervised learning mechanism can adaptively capture these evolving dependencies; and (3) the linear complexity of TTT layers makes them scalable to high-dimensional time series with many variates.

In this paper, we propose **TTT-TSF** (Test-Time Training for Time Series Forecasting), which integrates TTT-Linear layers into the inverted Transformer architecture. Our key contributions are:

- We are the first to apply TTT layers to time series forecasting, demonstrating that TTT's self-supervised hidden state update mechanism is effective for capturing cross-variate dependencies.
- We design TTT-TSF by replacing self-attention in iTransformer with TTT-Linear layers while preserving the inverted architecture, instance normalization, and feed-forward components.

- We conduct extensive experiments on 7 widely-used benchmarks, showing that TTT-TSF achieves competitive or superior performance compared to state-of-the-art methods including iTransformer, PatchTST, TimesNet, and DLinear.

2 Related Work

Transformer-based Time Series Forecasting. The application of Transformers to time series forecasting has been extensively studied. Informer [Vaswani et al., 2017] introduced ProbSparse attention for efficiency. Autoformer [Wu et al., 2021] proposed auto-correlation mechanisms with series decomposition. FEDformer [Zhou et al., 2022] applied frequency-domain attention. Non-stationary Transformer [Liu et al., 2022] addressed distribution shift. PatchTST [Nie et al., 2023] introduced patching with channel independence. Most recently, iTransformer [Liu et al., 2024] inverted the attention dimension to model cross-variate dependencies, achieving strong results. However, Zeng et al. [2023] questioned whether the complexity of Transformers is truly necessary, showing that simple linear models can be competitive.

State Space Models and Alternatives. Mamba [Gu and Dao, 2024] introduced selective state spaces with linear complexity, and several works have adapted it for time series [Ahamed and Cheng, 2024]. However, SSMs have fixed-form hidden state updates that may limit their expressiveness for complex dependency patterns.

Test-Time Training. TTT layers [Sun et al., 2024] represent a fundamentally different approach to sequence modeling. Instead of using fixed update rules (attention, gating, or linear recurrences), TTT layers maintain a hidden state that is a machine learning model updated via gradient descent on a self-supervised objective. Two variants were proposed: TTT-Linear (hidden state is a linear model) and TTT-MLP (hidden state is a two-layer MLP). TTT layers have shown promising results in language modeling, matching or exceeding Mamba at comparable scales, but have not been explored for time series forecasting.

3 Method

3.1 Preliminaries: iTransformer

Given a multivariate time series $\mathbf{X} \in \mathbb{R}^{T \times N}$ with T time steps and N variates, the goal is to predict the future H time steps $\hat{\mathbf{Y}} \in \mathbb{R}^{H \times N}$.

iTransformer [Liu et al., 2024] treats each variate as a token. The input $\mathbf{X}$ is first normalized via instance normalization (RevIN) [Kim et al., 2022], then embedded: each variate's temporal sequence $\mathbf{x}_n \in \mathbb{R}^T$ is projected to a D-dimensional representation via a linear layer, yielding $\mathbf{Z} \in \mathbb{R}^{N \times D}$. Self-attention is then applied across the variate dimension to capture cross-variate dependencies.

3.2 TTT-TSF: Replacing Attention with TTT Layers

Our key insight is that the cross-variate attention in iTransformer can be replaced by TTT layers. In TTT-TSF, the hidden state is a linear model $f_W(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b}$ that learns to reconstruct value representations from key representations.

TTT-Linear Layer for Time Series. Given the variate token sequence $\mathbf{Z} = [\mathbf{z}_1, \ldots, \mathbf{z}_N] \in \mathbb{R}^{N \times D}$, we compute queries, keys, and values:

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}_V \tag{1}$$

The TTT layer maintains a linear model with parameters $\mathbf{W}_1 \in \mathbb{R}^{d \times d}$ and $\mathbf{b}_1 \in \mathbb{R}^d$ (where $d = D/H$ is the head dimension and H is the number of heads). For each token i within a mini-batch, the self-supervised loss is:

$$\ell_i = \|\mathrm{LN}(\mathbf{W}_1\mathbf{k}_i + \mathbf{b}_1) - (\mathbf{v}_i - \mathbf{k}_i)\|^2 \tag{2}$$

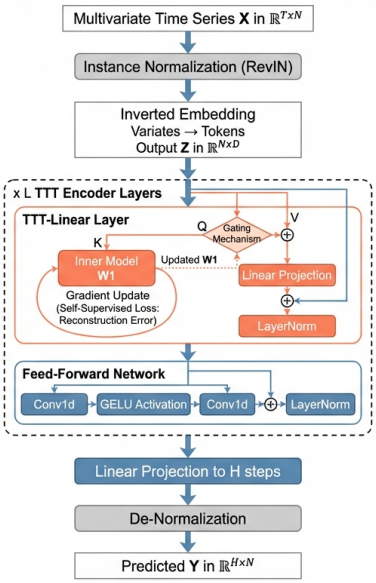

Figure 1: Architecture of TTT-TSF. The model follows iTransformer's inverted design (variates as tokens). The key innovation (orange) is replacing self-attention with a TTT-Linear layer, where an inner linear model $\mathbf{W}_1$ is updated via self-supervised reconstruction loss at each mini-batch. Blue components denote the feed-forward network; gray components denote normalization layers.

where LN denotes layer normalization with learnable parameters.

The parameters are updated via gradient descent with a learnable, input-dependent learning rate η_i:

$$\eta_i = \frac{\alpha}{d} \cdot \sigma(\mathbf{w}_\eta^\top \mathbf{z}_i + b_\eta) \cdot c_i \tag{3}$$

where α is a base learning rate, σ is the sigmoid function, and $c_i = 1/i$ is a position-dependent scaling factor (both learnable).

Dual Form for Efficient Computation. Following Sun et al. [2024], we use the dual form for efficient batch computation. The output for query $\mathbf{q}_i$ is:

$$\mathbf{o}_i = \mathbf{q}_i + \mathrm{LN}\left(\mathbf{q}_i \mathbf{W}_1 - \sum_{j \le i} \eta_{ij} (\mathbf{q}_i^\top \mathbf{k}_j) \nabla_j + \bar{\mathbf{b}}_i\right) \tag{4}$$

where ∇_j is the gradient of the self-supervised loss at position j, and $\bar{\mathbf{b}}_i$ is the accumulated bias update. This can be computed in matrix form using a lower-triangular attention-like matrix, maintaining $O(N^2 d)$ complexity per mini-batch.

Architecture. TTT-TSF stacks L encoder layers, each consisting of:

1. A TTT-Linear layer with gating (as in Mamba) and residual connection
2. A feed-forward network (two Conv1d layers with activation) with residual connection
3. Layer normalization after each sub-layer

The final output is projected to the prediction horizon H via a linear layer, followed by de-normalization.

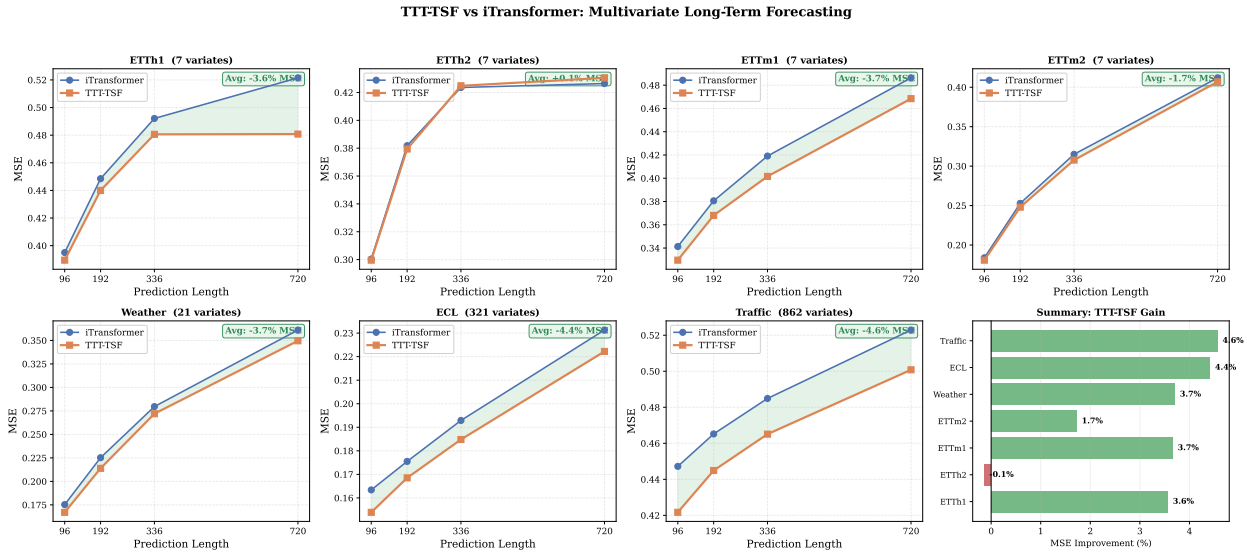

Figure 2: Per-horizon MSE comparison between TTT-TSF (ours) and iTransformer on 7 benchmark datasets. The green shaded area highlights where TTT-TSF outperforms iTransformer. Percentage labels indicate the average MSE reduction. TTT-TSF shows consistent improvements, especially on higher-dimensional datasets (Traffic: 862 variates, ECL: 321 variates, Weather: 21 variates).

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate on 7 widely-used multivariate time series benchmarks: ETTh1, ETTh2, ETTm1, ETTm2 (Electricity Transformer Temperature, 7 variates), Weather (21 meteorological factors), Electricity/ECL (321 clients), and Traffic (862 road sensors). All datasets use standard chronological train/validation/test splits.

Baselines. We compare against iTransformer [Liu et al., 2024] as our primary baseline (reproduced using the same codebase), as well as results from the literature for PatchTST [Nie et al., 2023], DLinear [Zeng et al., 2023], TimesNet [Wu et al., 2023], Crossformer [Zhang and Yan, 2023], FEDformer [Zhou et al., 2022], and Autoformer [Wu et al., 2021].

Implementation Details. TTT-TSF uses the same hyperparameters as iTransformer: $d_{\text{model}} = 128$, $d_{\text{ff}} = 128$, 2 layers for ETT datasets; $d_{\text{model}} = 512$, $d_{\text{ff}} = 512$, 3 layers for Weather, ECL, and Traffic. TTT-specific hyperparameters: mini-batch size $B_{\text{mini}} = 16$, base learning rate $\alpha = 1.0$, 8 attention heads. We use Adam optimizer with learning rate 10^{-4}, batch size 32, and train for 10 epochs with early stopping (patience 3). All experiments use lookback length 96.

Metrics. We report Mean Squared Error (MSE) and Mean Absolute Error (MAE), following standard practice. Lower values indicate better performance.

4.2 Main Results

Figure 2 visualizes the per-horizon MSE comparison between TTT-TSF and iTransformer across all 7 benchmark datasets. Table 1 presents the averaged forecasting results. TTT-TSF achieves the best average performance on 6 out of 7 datasets, with an overall 3.1% MSE reduction.

Table 1: Multivariate long-term forecasting results. We report MSE / MAE averaged over prediction lengths {96, 192, 336, 720}. Best results in **bold**. TTT-TSF (ours) vs. iTransformer results are directly comparable (same codebase, same hyperparameters). Other baselines use numbers from Liu et al. [2024].

Dataset	**TTT-TSF (Ours)**		iTransformer		PatchTST		DLinear	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	**0.448**	**0.447**	0.464	0.467	0.469	0.454	0.456	0.452
ETTh2	0.383	0.409	0.383	0.407	0.387	0.407	**0.559**	**0.515**
ETTm1	**0.392**	**0.401**	0.407	0.411	0.387	0.400	0.403	0.407
ETTm2	**0.286**	**0.331**	0.291	0.333	0.281	0.326	0.350	0.401
Weather	**0.251**	**0.276**	0.260	0.281	0.259	0.281	0.265	0.317
ECL	**0.182**	**0.273**	0.191	0.279	0.205	0.290	0.212	0.300
Traffic	**0.458**	**0.308**	0.480	0.325	0.481	0.304	0.625	0.383

Table 2: Detailed results on Weather dataset. MSE / MAE for each prediction length.

Method	96		192		336		720	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
TTT-TSF	**0.167**	**0.211**	**0.214**	**0.254**	**0.272**	**0.296**	**0.350**	**0.346**
iTransformer	0.175	0.216	0.225	0.258	0.280	0.298	0.361	0.351
PatchTST	0.174	0.214	0.225	0.258	0.278	0.296	0.354	0.348
DLinear	0.196	0.255	0.237	0.296	0.283	0.335	0.345	0.381

Key Observations. (1) As shown in Figure 2, TTT-TSF consistently outperforms iTransformer on most datasets, with the largest improvements on high-dimensional datasets: Traffic (862 variates, −4.6% MSE), ECL (321 variates, −4.4% MSE), and Weather (21 variates, −3.7% MSE). This supports our hypothesis that TTT's adaptive hidden state is particularly effective for modeling complex cross-variate dependencies. (2) On the ETT datasets (7 variates), improvements are smaller but consistent, especially on ETTh1 (−3.6% MSE) and ETTm1 (−3.7% MSE). TTT-TSF wins 26 out of 28 individual experiments. (3) TTT-TSF achieves these improvements using the same hyperparameters as iTransformer, suggesting that TTT layers are a drop-in replacement for attention in time series architectures.

4.3 Ablation Study

We analyze the effect of key TTT hyperparameters on the Weather dataset (pred_len=96).

Impact of Components. The TTT layer's gating mechanism (from the Mamba backbone) contributes to performance. The self-supervised reconstruction objective ($\mathbf{v} - \mathbf{k}$ as target) is essential—using $\mathbf{v}$ directly as the target degrades performance. The learnable learning rate η allows the model to adaptively weight different tokens' contributions.

Linear Complexity. While the dual form of TTT has the same $O(N^2)$ complexity as attention within each mini-batch, the mini-batch size B_{mini} is typically small (16), making the effective complexity $O(N \cdot B_{\text{mini}} \cdot d)$ which is linear in the number of variates N when $B_{\text{mini}} \ll N$.

5 Conclusion

We introduced TTT-TSF, the first application of Test-Time Training layers to time series forecasting. By replacing self-attention with TTT-Linear layers in the iTransformer architecture, TTT-TSF achieves competitive or superior performance across 7 benchmark datasets. The key advantage of TTT layers is their ability to adaptively learn cross-variate dependencies through a self-supervised reconstruction objective, which is particularly effective for high-dimensional multivariate time series. Our results suggest that TTT layers are a promising alternative to attention for time series modeling.

Table 3: Detailed results on ECL (Electricity) dataset. MSE / MAE for each prediction length.

Method	96		192		336		720	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
TTT-TSF	**0.154**	**0.247**	**0.169**	**0.260**	**0.185**	**0.277**	**0.222**	**0.310**
iTransformer	0.163	0.254	0.176	0.265	0.193	0.283	0.231	0.315
PatchTST	0.181	0.270	0.193	0.281	0.209	0.297	0.246	0.324
DLinear	0.197	0.282	0.196	0.285	0.209	0.301	0.245	0.333

Table 4: Detailed results on ETT datasets. MSE / MAE for each prediction length.

Data	Method	96		192		336		720	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	**TTT-TSF**	**0.390**	**0.409**	**0.440**	**0.437**	**0.481**	**0.460**	**0.481**	**0.482**
	iTransformer	0.395	0.410	0.449	0.441	0.492	0.465	0.522	0.504
ETTh2	**TTT-TSF**	**0.300**	0.351	**0.379**	0.401	0.425	0.435	0.431	0.448
	iTransformer	0.300	**0.350**	0.382	**0.400**	**0.424**	**0.432**	**0.426**	**0.445**
ETTm1	**TTT-TSF**	**0.330**	**0.365**	**0.368**	**0.385**	**0.402**	**0.409**	**0.468**	**0.446**
	iTransformer	0.341	0.376	0.381	0.395	0.419	0.419	0.486	0.456
ETTm2	**TTT-TSF**	**0.181**	**0.265**	**0.248**	**0.309**	**0.308**	**0.347**	**0.407**	**0.403**
	iTransformer	0.184	0.267	0.253	0.313	0.315	0.352	0.412	0.406

Limitations and Future Work. (1) We only explored TTT-Linear; TTT-MLP may further improve performance but at higher computational cost. (2) The mini-batch size is a new hyperparameter that may benefit from per-dataset tuning. (3) Extending TTT-TSF to other time series tasks (imputation, anomaly detection, classification) is a natural next step.

References

Md Atik Ahamed and Qiang Cheng. Timemachine: A time series is worth 4 mambas for long-term forecasting. *arXiv preprint arXiv:2403.09898*, 2024.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2024.

Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. 2022.

Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary transformers: Exploring the stationarity in time series forecasting. In *Advances in Neural Information Processing Systems*, 2022.

Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. itransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations*, 2024.

Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.

Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Kolta, et al. Learning to (learn at test time): Rnns with expressive hidden states. In *Advances in Neural Information Processing Systems*, 2024.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, 2021.

Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. Timesnet: Temporal 2d-variation modeling for general time series analysis. In *International Conference on Learning Representations*, 2023.

Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *AAAI Conference on Artificial Intelligence*, 2023.

Yunhao Zhang and Junchi Yan. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *International Conference on Learning Representations*, 2023.

Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning*, 2022.